

데이터셋 오토라벨링 프로젝트 — 기록

프로젝트 개요

- **프로젝트명:** 데이터셋 라벨링 자동화 (SAM 3 + YOLO Hybrid)
- **기간:** 8주
- **인원:** <AI 담당자> (AI 메인) + <백엔드 담당자> (백엔드·UI·인프라 메인)
- **GPU 서버 셋업 일자:** 2026.05.27
- **본 학습 완료 일자:** 2026.06.01
- **최신 업데이트 일자:** 2026.06.12 (V2~V4 학습, 진단, 데이터 보강 반영)

1. GPU 서버 정보 (매니저 제공)

항목	내용
서버 IP	<GPU_SERVER_IP>
호스트명	gpu-106
본인 계정	Jwson08
할당된 GPU	6번, 7번 (총 2장)
GPU 모델	NVIDIA RTX A6000 (확인됨)
환경변수 사용 방식	CUDA_VISIBLE_DEVICES=6,7 python ...
대안 옵션	SLURM (sbatch) 사용 가능
본인 작업 디렉토리	/home1/Jwson08/
서버 CUDA 버전	12.8
사용 가능 환경	기존 Qwen 작업 환경(Qwen3_VL 등) + 신규 환경 자유 생성 가능

RTX A6000 사양 참고:

- VRAM: 48 GB
- SAM 3(권장 16GB) + YOLO 학습 동시 실행 여유 충분
- 본 프로젝트에 충분한 성능 확보

2. VS Code Remote-SSH 연결

- VS Code의 Remote-SSH 확장으로 <GPU_SERVER_IP> 서버에 접속
- 이전 Qwen 프로젝트 작업 시 등록해둔 SSH 설정 재활용
- 통합 터미널(**Ctrl+**)을 통해 원격 서버에서 직접 작업

3. 기존 conda 환경 확인

명령어:

```
conda env list
```

결과:

```
base      * /home1/Jwson08/anaconda3
Qwen3_VL  /home1/Jwson08/anaconda3/envs/Qwen3_VL
Qwen3_VL_30B /home1/Jwson08/anaconda3/envs/Qwen3_VL_30B
Qwen3_VL_8B  /home1/Jwson08/anaconda3/envs/Qwen3_VL_8B
```

- 이전 Qwen 작업 환경 3개 존재 (그대로 유지)
- **autolabel** 환경 없음 확인 → 신규 생성 진행

환경 분리 정책: 이전 Qwen 환경을 재사용하지 않고 신규 환경을 만든 이유는 의존성 충돌 방지와 프로젝트별 환경 격리 원칙에 따름. Qwen 환경에는 transformers, accelerate 등 LLM 관련 무거운 패키지가 깔려 있어 본 프로젝트와 무관.

4. 신규 conda 환경 생성

4.1 환경 생성

명령어:

```
conda create -n autolabel python=3.10 -y
```

설정:

- 환경 이름: **autolabel**
- Python 버전: **3.10**
- 자동 승인(**y**) 플래그 사용

Python 3.10을 선택한 이유:

- PyTorch, Ultralytics, OpenCV 등 주요 라이브러리가 가장 안정적으로 지원하는 버전대
- 3.12, 3.13 같은 최신은 일부 패키지에서 빌드 오류 가능성
- 본인 PC 로컬 환경도 동일하게 3.10으로 통일 → 환경 일관성 확보
- 동기에게 전달한 온보딩 가이드와도 일치

결과: **Executing transaction: done** 으로 생성 완료

4.2 환경 활성화

명령어:

```
conda activate autolabel
```

확인: 터미널 프롬프트가 **(base)** → **(autolabel)**로 변경됨

4.3 패키지 설치

명령어:

```
pip install numpy opencv-python matplotlib pillow ultralytics
```

설치 대상:

- **numpy** — 배열 처리
- **opencv-python** — 이미지/영상 처리, polygon 그리기
- **matplotlib** — 결과 시각화
- **pillow** — 이미지 파일 입출력
- **ultralytics** — YOLO/SAM 모델 사용 wrapper (PyTorch 자동 동반 설치)

ultralytics 선택 이유:

- YOLO11/12 + SAM 2/3를 단일 패키지에서 모두 사용 가능
- 본 프로젝트의 두 핵심 모델을 같은 API로 다룰 수 있어 코드 단순화
- 별도 GitHub clone 없이 `from ultralytics import YOLO, SAM` 한 줄로 시작 가능
- 사전학습 weights 자동 다운로드 지원

5. PyTorch GPU 인식 문제 해결

5.1 초기 진단 — CUDA 인식 실패

검증 명령어:

```
CUDA_VISIBLE_DEVICES=6,7 python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

결과: **CUDA: False**

5.2 원인 분석

PyTorch 버전 확인:

```
python -c "import torch; print(torch.__version__)"
```

결과: **2.12.0+cu130**

원인 확정: ultralytics가 자동 설치한 PyTorch가 **CUDA 13.0 빌드(cu130)**인데, 서버 CUDA 드라이버가 **12.8**이라 호환 불가.

핵심 원리: PyTorch가 빌드된 CUDA 버전 ≤ 서버 드라이버 CUDA 버전이어야 함.

- **cu130** 요구 ≥ 13.0
- 서버 제공 = 12.8
- 요구 > 제공 → CUDA 인식 실패

5.3 해결 — PyTorch 재설치

기존 제거:

```
pip uninstall torch torchvision -y
```

CUDA 12.1 빌드로 재설치:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

cu121을 선택한 이유:

빌드	요구 CUDA	서버 호환 여부	비고
cu118	11.8 이상	12.8 호환	오래되어 일부 최신 기능 미지원
cu121	12.1 이상	12.8 호환	표준 선택, 자료 풍부
cu124	12.4 이상	12.8 호환	최신, 가능하지만 cu121이 더 검증됨
cu130	13.0 이상	12.8 미호환	서버 드라이버가 못 따라감

→ cu121이 **안정성·호환성·자료량**의 균형이 가장 좋음.

5.4 재검증 — 성공

CUDA: True
GPU count: 2
GPU name: NVIDIA RTX A6000

정상 인식. GPU 환경 셋업 완료.

6. 최종 환경 정보 요약

항목	값
서버	gpu-106 (<GPU_SERVER_IP>)
GPU	NVIDIA RTX A6000 × 2 (48GB VRAM × 2)
할당 GPU 번호	6, 7
서버 CUDA Driver	12.8
OS	Linux (서버)
conda 환경	autolabel
Python	3.10
PyTorch	2.x.x+cu121
핵심 패키지	numpy, opencv-python, matplotlib, pillow, ultralytics

7. 작업 디렉토리 생성

```
mkdir -p ~/autolabel/{data/images,data/labels,scripts,outputs,models}  
cd ~/autolabel
```

생성된 폴더 구조:

```
/home1/jwson08/autolabel/  
├── data/  
│   ├── images/  # CVAT 이미지 저장 위치  
│   └── labels/   # CVAT 라벨(.txt) 저장 위치  
├── scripts/      # 실행 스크립트  
├── outputs/      # 결과물 저장 (시각화, 추론 결과)  
└── models/       # 학습 모델 weights 저장
```

폴더 구조 설계 이유:

- **data/images**와 **data/labels** 분리 → Ultralytics YOLO 표준 데이터 구조와 일치
- **outputs/**를 별도 분리 → 입력 데이터와 결과물 혼재 방지, 정리 용이
- **scripts/**와 **models/** 분리 → 코드와 weights의 역할별 격리
- 로컬 Windows 환경과 동일한 구조 → 코드 이식성 확보

8. 로컬 코드 원격 서버 이관

8.1 이관 대상

파일	역할	크기
visualize_label.py	CVAT 라벨 시각화 검증 스크립트 (트랙 #2)	3KB
baseline_inference.py	사전학습 YOLO Baseline 추론 스크립트 (트랙 #3)	5KB

8.2 이관 방법

VS Code의 Upload 메뉴 사용:

1. 원격 VS Code Explorer에서 `~/autolabel/scripts` 폴더 우클릭
2. Upload... 선택
3. 로컬 `C:\Users\WHDCLABS\autolabel\scripts`의 두 파일 선택

처음에는 드래그앤드롭을 시도했으나 VS Code Remote-SSH에서 항상 안정적으로 동작하지 않음을 확인. Upload 메뉴 방식이 더 신뢰성 있음.

8.3 이관 확인

```
ls ~/autolabel/scripts/  
# 결과: baseline_inference.py visualize_label.py
```

두 파일 모두 이관 완료

9. 코드 환경 적응 — Linux 경로 + matplotlib 백엔드 변경

9.1 변경 ① — 파일 경로 (Windows → Linux)

경로 수정 핵심 변경점:

- 백슬래시(`\`) → 슬래시(`/`) — Linux 경로 컨벤션
- `C:\Users\WHDCLABS` → `/home1/Jwson08/` — Linux 홈 디렉토리
- `r""` (raw string) 제거 — Linux 경로엔 백슬래시 없어 불필요
- `OUTPUT_PATH`는 `data/`가 아닌 `outputs/` 폴더로 변경 — 입력/결과물 분리

9.2 변경 ② — matplotlib 백엔드를 Agg로 변경

```
import cv2  
import numpy as np  
import matplotlib  
matplotlib.use('Agg') # GUI 없는 원격 서버용 백엔드  
import matplotlib.pyplot as plt
```

`matplotlib.use('Agg')`가 필요한 이유:

- 원격 서버는 GUI(디스플레이)가 없어서 `plt.show()` 동작 불가
- 기본 백엔드(TkAgg, Qt5Agg 등)는 GUI를 요구해서 에러 발생
- `Agg`는 파일 출력 전용 백엔드 — 화면에 표시 안 하고 `plt.savefig()` 등으로 파일만 저장
- 원격 서버 작업 시 표준 패턴

위치 주의: `matplotlib.use('Agg')`는 반드시 `import matplotlib.pyplot as plt`보다 먼저 와야 함.

10. 트랙 #3 — GPU 환경 검증 (Baseline 추론 재실행)

10.1 CVAT 데이터 NAS 위치 확인

확인된 CVAT 데이터 구조:

```
/nas03/1_EV_LABELING/  
├── corner/   ← 95개 영상 폴더
```

```

|   └── GX010097_011/
|       ├── GX010097[1]_011_0000.jpg (이미지들)
|       ├── ...
|       └── labels/train/*.txt (라벨)
└── center/ ← 147개 영상 폴더

```

10.2 샘플 1쌍 복사

명령어 (대괄호 [1] 때문에 작은따옴표 필수):

```

cp '/nas03/1_EV_LABELING/corner/GX010097_011/GX010097[1]_011_0000.jpg' ~/autolabel/data/images/
cp '/nas03/1_EV_LABELING/corner/GX010097_011/labels/train/GX010097[1]_011_0000.txt' ~/autolabel/data/labels/

```

10.3 첫 실행 — 디바이스: cpu 문제

콘솔 출력:

```

모델: yolo11n-seg.pt
디바이스: cpu ← 문제
객체 1: person (conf=0.41, polygon 점 352개)

```

원인: `CUDA_VISIBLE_DEVICES=6,7`을 썼지만 코드에서 명시적으로 GPU 사용을 안 해서 ultralytics가 자동으로 CPU 선택.

10.4 수정 — device=0 인자 추가

```

results = model.predict(source=IMAGE_PATH, conf=0.25, device=0, verbose=False)

```

device=0의 의미:

- `CUDA_VISIBLE_DEVICES=6,7` 환경변수가 실제 GPU 6, 7번을 Python 입장에서 `cuda:0`, `cuda:1`로 매핑
- 코드의 `device=0`은 매핑된 첫 번째 GPU (= 실제 6번 GPU)

10.5 재실행 — 성공

```

디바이스: cuda:0 ← GPU 사용 확인
객체 1: person (conf=0.41, polygon 점 352개)

```

GPU 환경의 최종 검증 완료.

그래프/이미지 첨부 가이드:

Baseline 추론 결과 시각화 (`outputs/baseline_result.jpg`)

[이미지 첨부: ![스크린샷 2026-06-02 095354.png]]

트랙 #7 Before/After 비교에서 “Before” 자료로 다시 사용.

11. 트랙 #4 — SAM 3 동작 확인

11.1 HuggingFace 인증

SAM 3는 Meta 라이선스 동의를 필요한 게이트 모델.

진행 절차:

1. HuggingFace 계정으로 로그인
2. Meta SAM 3 페이지에서 라이선스 동의 신청 → 승인 받음
3. HuggingFace 토큰 발급 (Read 권한)
4. GPU 서버에서 `hf auth login` (이전 명령어 `huggingface-cli`는 deprecated)

토큰 등록 결과:

Login successful.
Token saved to /home1/Jwson08/.cache/huggingface/token

빨간색 git credential 경고는 무관 (모델 push할 때 필요한 것). 우리는 다운로드만 하므로 무시.

11.2 SAM 3 추론 스크립트 작성

Claude Code로 `scripts/sam3_click_inference.py` 작성:

- SAM 3 모델 자동 다운로드 (3.4 GB)
- 클릭 좌표 기반 segmentation
- mask → polygon 변환 (cv2.findContours + Douglas-Peucker)
- 결과 시각화

11.3 1차 시도 — 좌표 어긋남

수동으로 어림짐작한 좌표(1100, 450) / (800, 350)이 객체 위치와 안 맞음.

- 의도: 사람, 강아지
- 결과: 강아지, 바닥 잡힘 (이미지 회전 + 좌표계 오인)

11.4 2차 시도 — 라벨에서 자동 좌표 추출

수정 내용:

- CVAT 라벨 파일에서 polygon의 첫 점을 자동으로 클릭 좌표로 사용
- 정규화 좌표(0~1) → 픽셀 좌표 변환

결과:

- 강아지: 정밀한 polygon, 윤곽 잘 따라감
- 사람: 상체만 잡고 다리/발 누락

11.5 3차 시도 — Multi-point Prompt

수정 내용:

- 사람(class 0)에 점 2개 사용 (`labels=[1, 1]`)
- 강아지(class 1)는 기존 1점 유지
- Douglas-Peucker epsilon 2.0 → 1.5로 감소

결과:

- 다리/발 여전히 누락
- 원인 분석: SAM의 본질적 한계

1. 시각적 단절 — 흰 코트와 검은 다리/신발의 색상·질감 차이가 너무 큼
2. 가려짐(occlusion) — 웅크려 앉은 자세로 다리가 부분적으로 가려짐
3. 다리 점의 약한 반영 — 주변 픽셀이 상체와 달라 SAM이 별개 영역으로 판단

11.6 트랙 #4 결론

SAM의 본질적 동작 방식 때문이며, 코드/환경 문제 아님.

- SAM 3 동작 확인 (트랙 #4 목적 달성)

- 클릭 → mask → polygon 파이프라인 검증
- 강아지는 정말하게 잡음 (SAM 능력 입증)
- multi-point prompt 동작 검증
- [postprocess.py](#) 베이스 코드 확보

면접 자료 가치: “SAM이 시각적 연속성 기반으로 동작하므로, 시각적으로 분리된 복합 객체는 한 번의 클릭으로 잡기 어렵다는 한계를 확인. 실제 라벨링 도구에서는 multi-point 또는 box prompt로 보완하도록 설계함.”

그래프/이미지 첨부 가이드:

3차 시도 결과 이미지 ([outputs/sam3_click_result.jpg](#))

[이미지 첨부: ![image.png]]

다리 누락 문제를 시각적으로 보여주는 자료.

12. 트랙 #5 — 데이터셋 분할 (train/val/test)

12.1 데이터 규모 파악

전체 데이터 규모:

구분	영상 폴더	추정 프레임 (x62)
corner	95개	약 5,890장
center	147개	약 9,114장
합계	242개 영상	약 15,000장

12.2 corner/center 겹침 확인 (data leakage 방지)

```
comm -12 <(ls /nas03/1_EV_LABELING/corner/ | sort) <(ls /nas03/1_EV_LABELING/center/ | sort)
```

결과: 출력 없음 (corner와 center는 완전히 다른 영상)

→ 그냥 합쳐서 영상 단위로 분할 가능

12.3 분할 전략

핵심 원칙: 영상 단위 분할 (프레임 단위 random split 금지)

- 같은 영상의 인접 프레임은 거의 동일한 장면
- 프레임 단위로 random split 시 train과 val에 비슷한 프레임 섞임
- → **Data leakage** 발생, 평가가 비현실적으로 높게 나옴

올바른 방법: 영상 폴더 단위로 train/val/test 분배

- 한 영상의 모든 프레임은 한쪽 split에만 들어감
- 평가가 “본 적 없는 영상”에 대해 이뤄짐 → 진짜 일반화 성능 측정

분할 비율 (7:2:1):

Split	영상 수	용도
train	약 169개 (70%)	학습용
val	약 48개 (20%)	학습 중 성능 확인
test	약 25개 (10%)	최종 평가용

12.4 분할 스크립트 (split_dataset.py)

Claude Code로 작성. 주요 기능:

- corner와 center 각각 7:2:1로 분할 후 합치기 (분포 유지)
- random seed 고정 (재현성)
- Ultralytics 표준 구조로 심볼릭 링크 생성
- data.yaml 자동 생성

심볼릭 링크 선택 이유:

- 복사: 15,000장을 실제 복사 → 디스크 2배 사용, 시간 오래
- 심볼릭 링크: 원본을 가리키는 “바로가기”만 생성 → 디스크 거의 안 씀, 즉시

12.5 분할 결과 검증

split	이미지	라벨	일치
train	11,978장	11,978장	
val	3,891장	3,891장	
test	(소수)	동일	

12.6 data.yaml

```
path: /home1/jwson08/autolabel/dataset
train: images/train
val: images/val
test: images/test
names:
  0: person
  1: dog
```

13. 트랙 #6-A — 1차 검증 학습 (train_check)

13.1 검증 학습 전략

원칙: “전체 데이터 + 짧은 epoch”으로 파이프라인 검증

- 데이터는 그대로 (15,000장 전체)
- epoch만 10으로 단축
- 목적: 학습 코드/환경에 문제 없는지 빠르게 확인
- 정상 동작 확인 시 → 본 학습(epoch 100)으로 진입

13.2 검증 학습 설정

항목	값
모델	YOLO11s-seg
Epoch	10
Batch	16
imgsz	640
GPU	1개 (RTX A6000)
데이터	train 11,978장 / val 3,891장

왜 "YOLO11"인가 (버전 선택, 2026-06-12 보강):

버전	Segmentation 지원	비고
YOLOv8	pretrained seg 제공	2023.01 출시, 이전 세대
YOLOv9 / YOLOv10	탐지 중심	v10은 NMS 제거에 집중, 공식 instance segmentation 모델 미제공
YOLO11	pretrained seg 제공	2024.09 출시. seg/pose/classify/OBB를 단일 아키텍처로 지원
YOLO12	seg는 yaml만 제공	seg/pose 등은 처음부터 학습해야 함 — 공식 pretrained weight 無, Ultralytics도 production에는 YOLO11/YOLO26 권장

- YOLO11은 YOLOv8 대비 파라미터 22% 감소하면서도 정확도 향상 (YOLO11m: COCO mAP 95.0%, YOLOv8m보다 파라미터 22% 적음)
- C2f → C3k2 블록, C2PSA(Cross-Stage Partial Spatial Attention) 도입으로 작은 객체 탐지 성능 개선 — 본 프로젝트의 강아지(dog)처럼 상대적으로 작은 객체에 유리
- YOLOv10 대비 추론 속도 약 2% 빠름
- → 인스턴스 세그멘테이션을 공식적으로, 안정적으로(pretrained weight 포함) 지원하는 최신 버전이 YOLO11이라 선택

참고: [Ultralytics YOLO11 vs 이전 모델

비교](<https://www.ultralytics.com/blog/comparing-ultralytics-yolo11-vs-previous-yolo-models>), [YOLO11 vs YOLOv8](<https://docs.ultralytics.com/compare/yolo11-vs-yolov8/>), [YOLO12 공식 문서](<https://docs.ultralytics.com/models/yolo12>)

왜 사이즈 "s(small)"인가:

- n(nano) 6MB → 너무 가벼움, 정확도 한계
- s(small) 약 22MB → 정확도 + 속도 균형
- m(medium) 이상 → 시간 더 걸림, 본 학습에서 검토

13.3 검증 학습 결과 — Epoch별 mAP

Epoch	mAP50 (Mask)	mAP50-95 (Mask)
1	0.8231	0.8645
2	0.7961	0.8543
3	0.8115	0.8715
4	0.8106	0.8668
5	0.8353	0.8807
6	0.8352	0.8774
7	0.8549	0.9039
8	0.8412	0.8898
9	0.8590	0.8897
10	0.8703	0.9075

평가:

- 최종 mAP50 0.8703 — 매우 높은 수준
- 10 epoch에서도 상승 추세 유지 (수렴 안 됨)
- epoch이 진행되며 loss 감소, mAP 상승 패턴 정상
- → 본 학습으로 진행 결정

baseline 대비 비교 (트랙 #3 결과):

- baseline (사전학습 YOLO11n, 학습 안 함): person conf 0.41, dog 미감지
- 검증 학습 (10 epoch): mAP50 0.87
- → 도메인 파인튜닝의 효과가 정량적으로 입증됨

14. 트랙 #6-B — 2차 본 학습 (train_v1) 완료

14.1 본 학습 설정

항목	값
모델	YOLO11s-seg
Epoch	100 (완주)
Batch	32 (검증 학습의 16에서 증가)
imgsz	640
GPU	2개 (RTX A6000 × 2, DDP)
데이터	train 11,978장 / val 3,891장
학습 시간	약 1시간 20분 (4832초)

검증 학습 대비 변경 사항:

- GPU 2장 활용 (DDP) — 검증은 1장, 본 학습은 2장 병렬
- Batch 16 → 32 증가 — GPU 2장이자 메모리 여유, 학습 안정성·속도 향상
- Epoch 10 → 100 — 모델이 완전히 수렴할 때까지

DDP(Distributed Data Parallel) 동작 원리:

- 데이터를 두 GPU에 분배 (예: batch 32 → GPU당 16씩)
- 각 GPU가 forward·backward 동시 진행
- gradient를 통신해서 동기화 → weight 업데이트
- 단일 GPU 대비 약 1.7~1.9배 빠른 학습

14.2 본 학습 진행 — Epoch별 mAP (주요 구간)

Epoch	mAP50 (Mask)	mAP50-95 (Mask)	비고
1	0.8482	0.9051	초기 상승
2	0.8612	0.9155	초기 상승
3	0.7297	0.8286	DDP 워밍업 진동
4	0.7149	0.8121	DDP 워밍업 진동
5	0.8014	0.8459	회복 시작
~	~	~	점진적 상승
68	0.9309	(Mask 최고점 부근)	피크 도달
78	0.9274	~	안정화
99	~	~	수렴

14.3 Epoch 3~5 일시 하락 — 해석

현상: epoch 1~2에서 0.86까지 올라갔다가 epoch 3~5에서 0.71~0.80으로 떨어짐

원인: DDP 2 GPU 워밍업 구간

- 학습 초반에 gradient 동기화·learning rate scheduler가 안정되지 않은 상태
- 두 GPU의 gradient가 합쳐지면서 일시적 진동 발생
- batch 32로 키운 영향 — effective batch가 커지면 초반 불안정 가능
- 정상적인 학습 곡선의 일부

결과: epoch 6 이후 회복, epoch 50 이후 0.92 부근 안정화. 정상 패턴 확인.

14.4 본 학습 최종 결과

지표	값	평가
Box mAP50	~0.92	매우 우수
Box mAP50-95	~0.81	우수
Mask mAP50	~0.92	매우 우수
Mask mAP50-95	~0.78	우수
Precision (M)	~0.94	매우 높음
Recall (M)	~0.90	매우 높음

학습 곡선 분석 (results.png 기반):

- 모든 loss (box, seg, cls, dfl) 우하향 → 학습 정상 수렴
- mAP 곡선 우상향 후 수렴 → 적절한 학습량
- train-val 추세 일치 → **Overfitting 없음**, 일반화 성공

⇒ results.png

[이미지: ![스크린샷 2026-06-01 080353.png]]

[이미지: ![val_batch0_labels.jpg]]

val_batch0_labels.jpg

[이미지: ![val_batch0_pred.jpg]]

val_batch0_pred.jpg

15. Before/After 비교 및 모델 성능 평가 완료

본 학습으로 얻은 v1 모델의 성능을 baseline과 비교하고, 검증셋 전체에 대한 정량 평가 및 추론 속도 측정을 진행. 본 프로젝트의 첫 정량 성과 자료 확보.

15.1 작업 1 — 모델 정식 저장

학습 결과를 임시 위치(runs/segment/.../train_v1/)에서 버전 관리된 모델 디렉토리로 이전.

모델 버전 명명 규칙: yolo11s_seg_v{N}_{YYYYMMDD}/

저장 위치: ~/autolabel/models/yolo11s_seg_v1_20260601/

저장 파일:

best.pt	(20MB, 학습된 모델)
last.pt	(마지막 epoch 모델, 백업용)
README.md	(모델 메타 정보 + 성능 + 분석)

args.yaml (학습 설정, 재현성)
 results.png (학습 곡선)
 results.csv (epoch별 숫자)
 confusion_matrix.png (혼동 행렬)
 confusion_matrix_normalized.png (정규화 혼동 행렬)
 MaskPR_curve.png (Precision-Recall 곡선)
 MaskF1_curve.png (F1 곡선)
 predictions.json (검증셋 평가 결과, COCO 형식)

README.md에 모델 정보 + 학습 설정 + 성능 + 분석을 함께 기록 → 6개월 후에도 모델 이해 가능.

15.2 작업 2 — Before/After 단일 이미지 비교

같은 이미지(GX010097[1]_011_0000.jpg)에 baseline과 v1 각각 추론해서 시각·정량 비교.

baseline (트랙 #3 결과, 사전학습 YOLO11n):

모델: yolo11n-seg.pt
 디바이스: cuda:0
 객체 1: person (conf=0.41, polygon 점 352개 — 거칠고 self-intersection)
 dog: 미감지

Fine-tuned v1 (방금 학습된 YOLO11s-seg):

모델: yolo11s_seg_v1_20260601/best.pt
 디바이스: cuda:0
 이미지: GX010097[1]_011_0000.jpg
 감지된 객체 수: 2
 객체 1: person (conf=0.97, polygon 점 287개)
 객체 2: dog (conf=0.93, polygon 점 87개)

정량 비교:

항목	Baseline (YOLO11n 사전학습)	Fine-tuned v1	변화
모델 크기	6 MB	20 MB	—
person conf	0.41	0.97	+0.56 (2.4배)
dog 감지	미감지	conf 0.93	0 → 0.93
person polygon	352점 (self-intersection)	287점 (정밀)	정밀도 향상
dog polygon	—	87점	정밀 감지

첨부 권장: [outputs/baseline_result.jpg](#)와 [outputs/finetuned_result.jpg](#) 나란히 또는 위아래로

시각 결과 분석:

- 강아지 (dog 0.93): 빨간 옷을 입은 강아지 윤곽을 거의 완벽하게 따라감
- 사람 (person 0.97): 흰 코트 + 검은 머리 + 검은 다리/신발까지 모두 포함

15.3 SAM 한계와의 결정적 차이 (트랙 #4 vs 트랙 #7 비교)

트랙 #4에서 SAM 3는 사람의 다리/발을 잡지 못했음 (시각적 단절 + occlusion 때문). 그런데 파인튜닝된 YOLO11s-seg v1은 정확히 그 부분까지 잡음.

도구	사람 다리/발 감지
SAM 3 (학습 안 됨)	누락
Fine-tuned YOLO11s-seg v1	포함

원인 분석:

- SAM은 시각적 연속성 기반으로 동작 → 흰 코트와 검은 다리를 다른 영역으로 판단
- YOLO는 학습 데이터(CVAT 라벨)에서 “사람 = 머리+상체+다리+발 한 덩어리”라고 학습 → 시각적 단절을 학습으로 극복

면접 어필 포인트:

“SAM의 본질적 한계인 시각적 단절 문제를 학습 기반 YOLO11s-seg로 극복했습니다. SAM은 단일 클릭으로 사람의 흰 코트와 검은 다리를 같은 객체로 묶지 못했지만, 학습된 모델은 도메인 라벨에서 객체 경계를 학습해 정확히 분할합니다.”

15.4 작업 3 — 검증셋 전체 평가 (val 3,891장)

best.pt로 검증셋 전체 평가. 학습 중 평가는 매 epoch 빠르게 도는 것이고, 이건 학습 완료 후 정식 평가.

실행 명령어:

```
CUDA_VISIBLE_DEVICES=6,7 yolo segment val \
  model=models/yolo11s_seg_v1_20260601/best.pt \
  data=dataset/data.yaml \
  device=0 \
  imgsz=640 \
  batch=32 \
  save_json=True \
  project=evaluations \
  name=v1_val
```

공식 최종 성능표

Class	Box mAP50	Box mAP50-95	Mask mAP50	Mask mAP50-95
all	0.909	0.818	0.921	0.785
person	0.970	0.932	0.972	0.884
dog	0.849	0.705	0.871	0.687

분석:

Person 성능 — 거의 완벽

- Mask mAP50 0.972, mAP50-95 0.884
- mAP50-95가 0.88+ 라는 건 IoU 0.75 이상 같은 엄격한 기준에서도 매우 정밀하다는 의미
- SAM이 못 잡던 다리/발까지 정확히 학습됨

Dog 성능 — 양호하지만 person보다 낮음

지표	Person	Dog	차이
Mask mAP50	0.972	0.871	-0.101
Mask mAP50-95	0.884	0.687	-0.197

mAP50은 비교적 비슷한데 (0.97 vs 0.87) 더 엄격한 mAP50-95에서 격차가 큼. 즉 “강아지가 어디 있는지는 잘 알지만, 윤곽이 person만큼 정밀하지는 않음.”

원인 추정 (3가지):

1. 클래스 불균형 — 데이터셋에 person이 dog보다 훨씬 많을 가능성 (가장 큼)
2. 객체 크기 — dog가 person보다 작아서 픽셀 수 적음 → 정밀 segmentation 어려움
3. 모양 다양성 — dog는 자세·종·옷에 따라 변동 큼 (앉기/서기/쭈그리기)

V2 개선 방향:

- dog 데이터 추가 수집 (본 시스템 활용 가능)
- copy-paste augmentation
- 클래스별 loss 가중치 조정

첨부 권장: [evaluations/v1_val/](#) 안의 confusion_matrix, PR curve 등

15.5 작업 4 — 추론 속도(FPS) 측정

라벨링 도구에서 사용자 클릭 시 응답 속도의 정량 지표.

측정 방법:

- val 100장 무작위 샘플 (random.seed=42 고정)
- 워밍업 5장 (GPU 캐시 안정화)
- 본 측정 100장 (time.perf_counter로 개별 측정)

측정 결과:

```
모델: yolo11s_seg_v1_20260601/best.pt
디바이스: cuda:0
측정: val 100장 (워밍업 5장 제외)
---
평균: 49.1 ms
최소: 19.0 ms
최대: 119.6 ms
표준편차: 27.8 ms
---
FPS (이론치): 20.4 fps
```

분석:

지표	값	평가
평균 49.1 ms	평균 FPS 20.4	외부 요인 포함
최소 19.0 ms	52.6 FPS	단독 GPU 환경 추정치
표준편차 27.8 ms	평균의 57%	변동 큼

표준편차가 큰 이유:

- 공유 GPU 서버 환경 (gpu-106 다중 사용자)
- NAS 파일 I/O 변동 (캐시된 vs 새로 읽는 이미지)
- 일부 추론에서 다른 사용자 작업과 자원 경쟁

진짜 모델 성능 추정:

- 단독 사용 환경 가정 시 최소값 19ms 근처로 수렴 예상
- 약 50 FPS = 라벨링 도구에 즉시 응답 가능

라벨링 도구 사용성 평가:

FPS	사용성
60+	실시간 영상 처리 가능
30~60	라벨링 도구에 즉시 응답 우리 단독 환경 추정
10~30	사용 가능, 약간의 지연감
~10	답답함

15.6 트랙 #7 종합 결과

본 프로젝트의 첫 정량 성과:

항목	결과
모델	YOLO11s-seg v1 (yolo11s_seg_v1_20260601)
Mask mAP50 (전체)	0.921
Mask mAP50-95 (전체)	0.785
Person Mask mAP50-95	0.884 (매우 정밀)

항목	결과
Dog Mask mAP50-95	0.687 (양호, 개선 여지 있음)
baseline 대비 person	conf 0.41 → 0.97 (+0.56)
baseline 대비 dog	미감지 → conf 0.93
추론 속도 (단독 환경 추정)	약 20 ms / 50 FPS

16. Git 협업 환경 셋업 완료 (2026.06.01)

16.1 사전 확인 — 매니저/〈백엔드 담당자〉 답변

질문	답변
NAS 공유 경로	/nas03/1_EV_LABELING/git_repos/labeling-project.git
접속 서버	gpu-106 동일 (〈GPU_SERVER_IP〉)
Git 인증	file:// 비밀번호 없이 (NFS 마운트 rw)
모델 weights 경로	/nas03/models/labeling-project/

16.2 사전 검증

본인이 직접 테스트:

- NAS 쓰기 권한 확인 (mkdir, bare repo 생성 가능)
- file:// 인증 정상 동작 (비밀번호 없이 clone 성공)
- Git 버전 2.43.0 (〈백엔드 담당자〉 동일)

16.3 Git 사용자 설정

```
git config --global user.name "〈AI 담당자〉"
git config --global user.email "[회사 이메일]"
git config --global core.quotePath false
git config --global init.defaultBranch main
```

16.4 중앙 저장소 생성 (〈AI 담당자〉님이 1회)

```
mkdir -p /nas03/1_EV_LABELING/git_repos
cd /nas03/1_EV_LABELING/git_repos
git init --bare labeling-project.git
```

권한 확인: drwxrwxrwx (〈백엔드 담당자〉도 push 가능)

16.5 본인 작업 사본 + 폴더 구조

```
mkdir -p ~/work
cd ~/work
git clone /nas03/1_EV_LABELING/git_repos/labeling-project.git
cd labeling-project
mkdir ai backend frontend docker docs
```

16.6 .gitignore + README.md 작성

보강된 .gitignore (마스터플랜 v3 리스크 ③ 누락 항목 포함):

- 데이터/모델 (data/, dataset/, *.pt, *.jpg 등)
- Python (__pycache__/, .venv/, *.egg-info/)

- Jupyter (.ipynb_checkpoints/)
- Next.js (node_modules/, .next/, out/)
- 환경변수 (.env)
- IDE/OS (.vscode/, .DS_Store)

16.7 기존 AI 코드 이전

```
cp -r ~/autolabel/scripts ai/
```

이전된 코드:

- visualize_label.py (트랙 #2)
- baseline_inference.py (트랙 #3)
- sam3_click_inference.py (트랙 #4)
- split_dataset.py (트랙 #5)
- train.py (트랙 #6)
- eval.py
- finetuned_inference.py (트랙 #7)
- measure_inference_speed.py (트랙 #7)
- slurm_train_resume.sh

16.8 첫 커밋 + push

```
git add .
git commit -m "chore: 프로젝트 초기 구조 + AI 모듈 이전"
git push origin main
```

결과: 16 files changed, 1,232 insertions(+)

16.9 모델 weights NAS 복사

```
mkdir -p /nas03/models/labeling-project
cp ~/autolabel/models/yolo11s_seg_v1_20260601/best.pt /nas03/models/labeling-project/elevator_v1.pt
cp ~/autolabel/models/yolo11s_seg_v1_20260601/README.md /nas03/models/labeling-project/elevator_v1 README.md
```

모델 명명 규칙:

- 운영용 단순한 이름: **elevator_v1.pt** (<백엔드 담당자> .env 편의)
- 메타 정보: **elevator_v1.README.md** (관리용)

16.10 <백엔드 담당자>에게 셋업 완료 알림

clone 경로 + 폴더 구조 + 모델 위치 + 다음 협업 단계(Predictor 명세 회의 요청) 전달.

17. Predictor 인터페이스 명세 작성 완료 (2026.06.02)

17.1 명세 작성 과정

1. <AI 담당자>님 명세 초안 작성 (v1.0)
2. <백엔드 담당자>과 회의 진행
3. <백엔드 담당자>이 v2.1로 보강
4. Git 충돌 발생 → 해결 (v2.1 변경사항 흡수)

5. 최종 v2.0으로 push

17.2 명세 핵심 내용 (v2.0)

클래스 시그니처:

```
class Predictor:
    def __init__(self, model_path: str, sam_model_path: str = None)

    def predict(
        self,
        image_path: str,
        mode: str = "yolo",
        conf_threshold: float = 0.25,
        click_points: list = None,
        class_filter: list = None
    ) -> dict
```

3가지 모드:

- **yolo**: 자동 감지 (이미지 통째로)
- **sam**: 사용자 클릭 기반 (W5 본격 구현)
- **hybrid**: YOLO + SAM (W5 본격 구현)

Mock 모드 (<백엔드 담당자> v2.1 보강):

- **model_path="mock"**이면 가짜 응답 반환
- 백엔드 개발 시 모델 로딩 없이 빠른 개발 가능

출력 형식:

```
{
  "objects": [
    {
      "class_id": int,
      "class_name": str,
      "confidence": float,
      "bbox": [x1, y1, x2, y2],
      "polygon": [[x, y], ...]
    }, ...
  ],
  "image_size": {"width": int, "height": int},
  "metadata": {
    "mode": str,
    "model_version": str,
    "inference_time_ms": float,
    "num_objects_detected": int
  }
}
```

에러 처리:

- 이미지 못 읽음 → **FileNotFoundError**
- 모델 로드 실패 → **RuntimeError**
- SAM 모드 + click_points 없음 → **ValueError**
- 감지 객체 없음 → 빈 objects 리스트 (예외 X)
- 잘못된 mode → **ValueError**

클래스 ID 매핑:

- 0: person (MVP 학습됨)
- 1: dog (MVP 학습됨)
- 2: robot (V2 학습 예정)
- 3: wheelchair (V2 학습 예정)

17.3 명세 변경 절차

본 명세는 양쪽 협업의 기반이므로 **일방적 변경 금지**:

1. 변경 사유 + 영향 범위 제안
2. 양쪽 합의 후 문서 수정
3. 양쪽 각자 영역 코드 수정 → push
4. 통합 테스트로 검증

변경 이력:

일자	버전	변경 내용
2026-06-02	v1	<AI 담당자>님 초안 작성
2026-06-02	v2.1	<백엔드 담당자> 보강 (Mock 모드 추가)
2026-06-02	v2.0	최종 합의 push

18. ai/inference.py 1차 구현 완료 (2026.06.02)

18.1 구현 범위 (W2 1차)

기능	상태
mode="yolo"	정상 동작 (elevator_v1 사용)
model_path="mock"	Mock 응답 반환
mode="sam"	NotImplementedError (W5 본격 구현)
mode="hybrid"	NotImplementedError (W5 본격 구현)
class_filter	명세대로 동작
에러 처리 5가지	명세대로 동작

18.2 핵심 설계 결정

항목	결정	이유
<code>_mock = (model_path == "mock")</code>	init 시 플래그로 저장	predict마다 비교하는 것보다 init 1회 판단이 명확
<code>_YOLO_CLASSES = {0, 1}</code>	set으로 정의	in 연산 O(1), 명세 §4 “2,3 자동 필터” 구현
인자 검증을 <code>t_start</code> 이후	타이머 진입 직후 검증	검증 실패도 predict 호출로 봐서 <code>inference_time_ms</code> 에 포함
<code>os.path.exists</code> → <code>FileNotFoundError</code>	predict 내에서 직접 확인	명세 §6 에러 처리 테이블 그대로 구현
<code>raise NotImplementedError</code> (sam/hybrid)	빈 구현 대신 명시적 예외	W5 전에 백엔드가 호출하면 즉시 인지 가능
<code>round(conf, 4)</code>	float 소수점 4자리	JSON 직렬화 시 부동소수점 노이즈 제거
<code>polygon: [[int(x), int(y)] for ...]</code>	float → int 변환	명세 §5: polygon은 픽셀 좌표
<code>_mock_response</code> 별도 메서드	predict에서 분리	Mock 로직과 실제 추론 흐름 분리

18.3 검증 결과

정상 동작 테스트:

```
Objects: 2
person: conf=0.97, polygon=287점
dog: conf=0.93, polygon=87점
```

```
Image size: {'width': 1920, 'height': 1080}
Metadata: {'mode': 'yolo', 'model_version': 'elevator_v1', ...}
```

에러 케이스 통과:

- 없는 이미지 → FileNotFoundError
- SAM 모드 호출 → NotImplementedError
- 잘못된 mode → ValueError

class_filter 동작:

- [0] → person만 반환
- [1] → dog만 반환
- None → 전체 반환

18.4 push 완료

```
git add ai/inference.py
git commit -m "feat(ai): Predictor 클래스 1차 구현 (YOLO + Mock 모드)"
git push origin main
```

〈백엔드 담당자〉에게 알림 전달:

- git pull로 받기
- Mock 사용 예시 + 진짜 모델 전환 예시
- 다음 주 통합 테스트 일정 협의 요청

19. 1차 통합 테스트 통과 완료 (2026-06-02 오전)

배경: 이전 섹션(18)에서 ai/predictor.py 1차 구현 + push까지 완료. 〈백엔드 담당자〉이 백엔드와의 통합 여부를 8단계 자동 검증 스크립트로 확인.

19.1 검증 결과 (1차)

단계	내용	결과
STEP 1	import 확인	FAIL
STEP 2~7	추론 동작	미진행
STEP 8	백엔드 연동	FAIL

원인: ai/__init__.py 누락 + docstring 파일명 불일치(inference.py 잔재)

19.2 즉시 수정

수정 내용:

- ai/__init__.py 추가 (from ai.predictor import Predictor)
- predictor.py docstring: ai/inference.py → ai/predictor.py
- 〈백엔드 담당자〉이 AIPredictorAdapter 추가 → ai/predictor.py 픽셀 좌표 출력을 백엔드 정규화 좌표로 변환 담당

19.3 검증 결과 (2차)

단계	내용	결과
STEP 1~8	전체	모두 OK

의미: Mock-First Development 전략 첫 번째 실전 검증. 본인 코드가 운영 시스템에 실제로 통합됐음을 확인.

20. 명세 v2.1 합의 완료 (2026-06-02 오후)

배경: SAM 모드 구현 전, SAM 결과 객체에 class_id를 어떻게 전달할지 결정 필요. 두 가지 방안이 충돌.

20.1 옵션 비교

항목	옵션 A	옵션 B
방식	click_points에 class_id 포함 [x, y, label, class_id]	별도 인자 click_class_id
장점	점마다 다른 클래스 지정 가능 (확장성)	UI 흐름과 일치, 기존 백엔드 구조 유지
단점	백엔드 UI 흐름과 불일치	단일 세션 단일 클래스로 제한
추천	<AI 담당자>	<백엔드 담당자>

20.2 <백엔드 담당자> 근거

- 프론트엔드 UI 흐름: “한 SAM 세션 = 한 클래스 선택 후 클릭”
- backend/api/sam.py 기존 구조(class_id 분리)와 일치

결론: 옵션 B 합의 → 명세 v2.0 → v2.1 업데이트

20.3 변경된 predict 시그니처

```
def predict(
    self,
    image_path: str,
    mode: str = "hybrid",
    conf_threshold: float = 0.45,
    click_points: list = None,    # [[x, y, label], ...]
    click_class_id: int = 0,     # ← v2.1 추가
    class_filter: list = None,
) -> dict:
```

협업 교훈: 본인 직관(확장성)보다 동료의 현실(UI 흐름, 기존 구조)이 더 타당할 때가 있다. 좋은 근거를 따르는 자세가 좋은 인터페이스를 만든다.

21. SAM 실제 구현 완료 (2026-06-02 오후)

21.1 신규 파일: ai/sam_inference.py

트랙 #4(ai/scripts/sam3_click_inference.py) 코드를 클래스 형태로 캡슐화.

```
ai/
├── predictor.py    ← Predictor 클래스 (SAM/hybrid 통합)
├── sam_inference.py ← SAMPredictor 클래스 (신규)
├── scripts/
│   └── sam3_click_inference.py (원본 스크립트, 참고용)
```

21.2 SAMPredictor 핵심 로직

```
class SAMPredictor:
    def predict_with_clicks(self, image_path, click_points, class_id=0):
        pts = [[cp[0], cp[1]] for cp in click_points]
        labels = [cp[2] for cp in click_points]
        # SAM 추론 → mask → cv2.findContours → approxPolyDP → polygon 반환
```

21.3 ai/predictor.py 추가 메서드

메서드	역할
<code>_predict_sam</code>	SAM 단독 추론, SAMPredictor에 위임
<code>_predict_hybrid</code>	YOLO bbox 중심 → SAM 클릭 → polygon 교체
<code>_ensure_sam_loaded</code>	SAM lazy 로딩 (첫 호출 시에만 로드, 메모리 절약)

21.4 명세 v2.1 일치 검증

항목	명세	구현	일치
polygon 좌표	픽셀	<code>[[int(x), int(y)]]</code>	
SAM confidence	1.0 고정 또는 score	argmax 선택 후 round(4)	
빈 click_points	빈 리스트 반환	<code>if not click_points: return []</code>	
class_id 범위 밖	ValueError	<code>if class_id not in range(4)</code>	

22. 트러블슈팅 메모 (재발 방지용)

사건 ① ultralytics 자동 설치 시 PyTorch 버전 불일치

- 증상: `torch.cuda.is_available() = False`
- 해결: cu121 빌드로 재설치

사건 ② VS Code Remote-SSH 드래그앤드롭 미동작

- 해결: Upload 메뉴 또는 scp 사용

사건 ③ 원격 서버 matplotlib GUI 출력 불가

- 해결: `matplotlib.use('Agg')`를 pyplot import 이전에

사건 ④ baseline 추론이 GPU 아닌 CPU로 동작

- 해결: `model.predict(..., device=0)` 명시

사건 ⑤ SAM 3 다리/발 누락

- 원인: SAM의 시각적 연속성 기반 한계
- 교훈: 도구 한계를 데이터로 검증

사건 ⑥ DDP 학습 초반 mAP 일시 하락

- 원인: gradient 동기화·LR scheduler 안정화 전
- 해결: 정상 패턴이라 대기

사건 ⑦ results.csv 컬럼 매칭 혼동

- 증상: lr/pg0(0.002)를 mAP50-95로 잘못 인식
- 해결: 컬럼 번호 매겨서 확인

사건 ⑧ 학습 출력 폴더 경로 중첩

- 증상: runs/segment/runs/segment/train_v1/
- 해결: project 인자에 절대경로 명시

사건 ⑨ yolo segment val 결과 경로 중첩

- 증상: 평가 결과가 예상 위치에 없음
- 해결: find로 위치 찾은 뒤 모델 폴더로 복사

사건 ⑩ FPS 측정 시 표준편차 큼

- 원인: 공유 GPU 서버 환경 + NAS 파일 I/O 변동
- 교훈: 평균 + 최소값 함께 보고

사건 ⑪ Git 첫 협업 충돌 (<AI 담당자>님 push 시점에 <백엔드 담당자>도 push)

- 증상: ! [rejected] main -> main (fetch first)
- 원인: Git이 <백엔드 담당자> 작업물 보호
- 해결: git pull origin main → 자동 병합 → git push
- 교훈: 작업 시작 시 + push 전 항상 git pull 습관

사건 ⑫ Predictor 명세 v2.0 vs v2.1 충돌

- 증상: <백엔드 담당자>이 v2.1로 보강한 사이 <AI 담당자>님이 v2.0 push 시도
- 해결: 충돌 해결로 v2.1 변경사항 흡수 후 v2.0으로 push
- 검증: git log --oneline -5 docs/predictor_interface.md로 <백엔드 담당자> commit 이력 확인
- 교훈: 명세서 같은 공유 문서는 변경 후 양쪽 확인 필요

사건 ⑬ SAMPredictor 자동 다운로드 차단

- 증상: FileNotFoundError: SAM 모델 파일 없음: sam3_b.pt
- 원인: __init__에 os.path.exists 검증을 추가했더니 ultralytics의 자동 다운로드 동작을 막아버림
- 해결: 검증 로직 제거, ultralytics에 모델 로드 위임 (try/except로 RuntimeError만 감싸기)
- 교훈: 외부 라이브러리의 자동 동작을 막는 안전장치는 오히려 해가 됨

사건 ⑭ SAM 3 인증 다운로드 실패 → SAM 2 전환

- 증상: [Errno 2] No such file or directory: 'sam3_b.pt'
- 원인: SAM 3(sam3_b.pt)는 HuggingFace 인증 필요. 서버 환경에서 자동 다운로드 불가
- 해결: SAM 2(sam2_b.pt)로 임시 전환. SAM 2는 ultralytics 표준 모델로 자동 다운로드 가능
- 추가 발견: 이미 다운로드된 sam3.pt가 서버에 있어 절대 경로 지정으로도 해결 가능
- 교훈: 본 작업의 핵심은 SAM 통합 검증. 모델 버전은 운영 단계 선택. SAM 1/2/3 어느 것이든 인터페이스 동일

23. 트랙 #8 — V2 학습 (Dog 성능 개선) 완료

V1 평가(트랙 #7, §15.4)에서 발견된 dog 클래스의 mAP50-95 부족(0.687) 을 개선하기 위해 V1 best.pt에서 이어서 추가 학습 진행.

23.1 V2 학습 설정 (V1 대비 변경)

항목	V1	V2	변경 이유
시작 가중치	COCO pretrained	V1 best.pt	처음부터 다시 학습할 필요 없이 이어서 미세조정
Epoch	100	150	추가 수렴 여유 확보
Batch	32	32	동일
GPU	RTX A6000 × 2 (DDP)	RTX A6000 × 2 (DDP)	동일

23.2 V2 최종 성능 (val set)

지표	V1	V2	변화
Mask mAP50 (all)	0.921	0.917	-0.004
Mask mAP50-95 (all)	0.785	0.781	-0.004

23.3 V2 배포 — 운영 모델로 채택

```
cp best.pt /nas03/models/labeling-project/elevator_v2.pt
cp README.md /nas03/models/labeling-project/elevator_v2.README.md
```

- backend/config.py의 기본 model_path가 elevator_v2.pt를 가리키도록 설정 → 현재(2026-06-12 기준) 운영 중인 모델
- V1 대비 전체 mAP는 근소하게 낮아졌으나, dog 클래스 단독 성능은 개선되어 V1의 핵심 약점(§15.4) 보완 목적은 달성

24. 파이프라인 진단 (Step 2~5) 완료 (2026-06-08)

선배 피드백에 따라, 라벨링 파이프라인의 문제 원인을 YOLO / SAM / 통합 파이프라인 세 구간으로 나누어 정량 진단 진행. 진단 이미지: GX010097[1]_011_0000.jpg.

24.1 진단 결과 요약

구간	진단 결과
YOLO 탐지 정확도	정상 (conf person=0.96, dog=0.92, mAP@0.5=0.9175)
YOLO 경계(polygon) 정밀도	문제 — mask_ratio=4로 학습되어 polygon이 약 300점으로 거칠게 출력, mAP@0.5:0.95=0.7786

구간	진단 결과
SAM 3	정상 동작 — polygon 34~40점 (YOLO 대비 약 88% 점 수 감소, 더 매끈함)
Hybrid 모드	품질은 우수하나 추론 21초 — SAM 이미지 인코딩을 매 요청마다 재실행하는 게 원인

24.2 진단 수치 상세

항목	YOLO	SAM (bbox 기반)
person polygon 점 수	300점	34점
dog polygon 점 수	74점	35점
IoU (YOLO vs SAM) — person	—	0.9290
IoU (YOLO vs SAM) — dog	—	0.8716

추론 시간 비교:

모드	시간
YOLO 단독	1,289 ms
SAM 클릭 단독	1,088 ms
Hybrid	21,181 ms

24.3 즉시 수정한 버그

- `sam_inference.py`의 `_ensure_image_cached`에서 이중 인코딩 발생 → 불필요한 GPU 메모리 사용(OOM 유발 가능성) → 수정 완료

24.4 미해결 항목 (<백엔드 담당자> 협업 필요)

- ~~SAM3 vs SAM2 비교 (NAS에 sam2.pt 미보유)~~ → §24.6에서 해결
- Hybrid 모드 기본값 정책 결정
- SAM Warm-up(서버 기동 시 이미지 인코딩 선행) 구현

24.5 결론 — V3 학습 방향 확정

"YOLO 탐지(클래스/위치)는 정상이며, polygon 경계가 거친 문제는 `mask_ratio=4` 설정 때문이다. → `mask_ratio=1`(풀 해상도 마스크)로 V3 재학습하여 경계 정밀도를 개선한다."

24.6 후속 조치 — SAM3 → SAM2 교체 완료 (2026-06-09)

진단(§24.1~24.4)에서 미해결로 남았던 SAM3 vs SAM2 비교를 진행하고, **SAM3 → SAM2로 교체**.

교체 이유:

- SAM3는 메모리 사용량이 크고, prompt 입력 방식(인터페이스) 자체가 복잡해서 연동하기 어려움
- SAM2는 메모리 사용량이 작고 prompt 인터페이스가 단순해 연동이 쉬움 → polygon 품질은 동등한 수준 유지

결과:

- 모델 교체: `sam3.pt` (3.3GB) → `sam2_b.pt` (155MB)
- Peak GPU 메모리: 2280MB → 778MB (약 71% 감소)
- polygon 품질: 동등 (큰 차이 없음)
- NAS 경로: `/nas03/models/labeling-project/sam2_b.pt`

- 관련 commit: [labeling-project](#) 저장소 6d0e200 (feat: SAM3 → SAM2 교체 (OOM 해결, 메모리 71% 감소))
- 같은 날 `_ensure_image_cached` 이중 인코딩 버그 (§24.3)도 함께 정리 — 첫 호출 시 `model.predict()`로 초기화된 경우 `set_image()`를 추가 호출하지 않도록 수정

상세 변경 이력 및 배경은 [labeling-project](#) 저장소(<백엔드 담당자> 협업 문서) 참고.

25. 트랙 #9 — V3 학습 (`mask_ratio=1`, 풀 해상도 마스크) 완료

진단 (§24)에서 확인된 polygon 경계 정밀도 문제를 해결하기 위해, V2 best.pt에서 이어서 `mask_ratio=1`로 재학습.

25.1 V3 학습 설정 (V2 대비 변경)

항목	V2	V3	변경 이유
시작 가중치	V1 best.pt	V2 best.pt	가장 최신 모델에서 이어서
<code>mask_ratio</code>	4 (기본값)	1	핵심 변경 — 풀 해상도 마스크로 polygon 경계 정밀도 개선
<code>imgsz</code>	640	640	탐지 정확도는 이미 충분하여 유지
Batch	32	8	<code>mask_ratio=1</code> 은 마스크 해상도가 커져 메모리 사용량 증가 → 배치 축소
GPU	2	2 (DDP)	동일
Epoch	150	100	—

25.2 실행 — Slurm Job 22711

- 파일: `scripts/train_v3.py`, `scripts/slurm_train_v3.sh`
- 실행 위치: gpu-106
- epoch당 소요: 약 382초 (6.4분)
- 시작: 2026-06-08 08:44경 → 완료: 2026-06-08 19:30경 (총 100 epoch 완주)

25.3 V3 이전 실패 이력 (트러블슈팅)

Job ID	문제	원인	해결
22522	DDP NCCL timeout	<code>imgsz=1280</code> + DDP 조합 불안정	<code>imgsz=640</code> 으로 복귀
22537	<code>FileNotFoundError</code>	깨진 심볼릭 링크	링크 재생성
22666 / 22670	OOM	<code>imgsz=1280</code> + <code>batch=8</code> 메모리 초과	<code>batch=4</code> 로 축소 후 재시도
22671	(수동 종료)	진단 작업 (§24) 집중을 위해 <code>scancel</code>	—

25.4 V3 최종 성능 (val set)

지표	V2 (운영 중)	V3	변화
Mask mAP50 (all)	0.917	0.913	-0.004
Mask mAP50-95 (all)	0.781	0.774	-0.007

평가:

- **mask_ratio=1** 적용으로 polygon 출력 자체의 해상도/표현력은 개선되었으나, 검증셋 전체 mAP 수치는 V2 대비 소폭 하락
- **batch=8**로 줄어든 영향(GPU당 4장)도 있을 수 있음 — V3는 GPU당 약 3.65GB만 사용해 메모리 여유가 매우 큰 상태로 학습됨 (48GB 중 3.65GB, 약 7.6%)

결론: V3는 V2 대비 성능 개선이 없어 운영 모델은 V2(**elevator_v2.pt**)로 유지.

- → 데이터 보강 + 배치 확대로 V4 재시도 결정 (§26, §27)

26. 데이터 보강 — GT 667장 추가 완료 (2026-06-12)

기존 라벨링 결과물(GT) 11개 시퀀스, 총 667장의 이미지/라벨 쌍을 학습셋에 추가 반영.

26.1 검증 단계

- **incoming/images/*/** 및 **incoming/labels/*/**의 11개 GT 시퀀스 전수 확인
- 이미지 수 = 라벨 수 = **667** (시퀀스별 1:1 매칭 확인)
- 라벨 포맷 검증: 667개 파일 전체에서 **포맷 오류(bad line)** 0건
- 기존 train 데이터와의 파일명 충돌 검사: **충돌 0건**
- 디스크 여유 공간 확인: 3.9TB 여유 (충분)

26.2 병합

```
# images/labels 667쌍을 train으로 복사
cp incoming/images/*/*.jpg dataset/images/train/
cp incoming/labels/*/*.txt dataset/labels/train/
```

구분	병합 전	병합 후
train 이미지	11,977장	12,644장
train 라벨	11,977개	12,644개

- **dataset/labels/train.cache, val.cache** 삭제 → 다음 학습 시 새 데이터 포함하여 캐시 재생성

27. 트랙 #10 — V4 학습 (V3 + 데이터 보강, 4-GPU DDP) 진행중

V3 best.pt를 시작점으로, 보강된 train 12,644장 데이터로 추가 학습. 동시에 V3에서 확인된 **GPU 메모리 여유(3.65GB/48GB)**를 활용해 batch를 대폭 확대.

27.1 V4 학습 설정 (V3 대비 변경)

항목	V3	V4	변경 이유
시작 가중치	V2 best.pt	V3 best.pt	최신 모델에서 이어서
데이터	train 11,977장	train 12,644장 (+667, §26)	데이터 보강
Batch	8	64	V3가 GPU당 3.65GB만 사용 → 8배 확대 (GPU 4장 × 16장)
GPU	2 (DDP)	4 (DDP)	배치 확대에 따라 GPU 추가 할당
mask_ratio	1	1 (동일)	V3의 핵심 개선사항 유지
imgsz	640	640	동일

항목	V3	V4	변경 이유
Epoch	100	100	동일

27.2 실행 — Slurm Job 23307

- 파일: `scripts/train_v4.py`, `scripts/slurm_train_v4.sh`
- 실행: gpu-106, GPU 0~3 (4장)
- GPU 사용률: 93~100%
- GPU 메모리: 약 10.7~14.8GB / 48GB per GPU (약 22~31%, 여전히 여유)
- epoch당 소요: 약 160~170초
- 예상 완료: 학습 시작 후 약 4.5~4.7시간

27.3 V4 최종 성능 (val set) 완료 (2026-06-12)

100 epoch 완주 후 `eval.py`로 val set 전체 재평가.

지표	V1	V2 (운영중)	V3	V4
Box mAP50	-	-	-	0.9100
Box mAP50-95	-	-	-	0.8153
Mask mAP50	0.921	0.917	0.913	0.922
Mask mAP50-95	0.785	0.781	0.774	0.7715

- Mask mAP50(핵심 지표)은 V4가 전 버전 중 최고치
- 다만 Mask mAP50-95는 V2(0.781) 대비 -0.0071로, §28.1 배포 조건 미달

27.4 mAP50-95(M)이 V1/V2 수준으로 회복되지 않는 이유

V2/V3/V4의 `results.csv`에서 epoch별 Mask mAP50-95 추이를 비교한 결과, 세 버전 모두 동일한 패턴을 보임 — 학습 초반(20~54epoch)에 정점을 찍고, 이후 100(~130)epoch까지 진행하면서 오히려 하락.

모델	정점 epoch	정점 값	최종 epoch 값
V2 (150ep)	30	0.781	100ep: 0.774 / 130ep: 0.771
V3 (100ep)	54	0.774	0.749
V4 (100ep)	30	0.775	0.758

추정 원인

- 매 버전이 이전 `best.pt`의 가중치만 이어받고, optimizer-LR은 `lr0=0.01` (MuSGD)부터 새로 시작 → 이미 정밀하게 맞춰진 마스크 경계가 초반 큰 LR에 흔들림 ("워밍업 충격")
- mAP50-95는 경계 정밀도에 민감한 엄격한 지표라, 100epoch 이상의 장기 학습에서 overfitting 영향을 크게 받음
- V1 → V2 → V3 → V4로 이어지는 continual fine-tuning 구조상 매 사이클마다 이 현상이 반복되어 0.77~0.78대에서 천장 형성

28. 최종 결론 — V2 유지, 추가 학습 보류 (2026-06-12)

§28.1(구) 배포 조건(V2의 Mask mAP50-95=0.781 능가)을 V4가 충족하지 못함에 따라 다음과 같이 결정.

- 운영 모델: V2(`elevator_v2.pt`) 유지 — `backend/config.py` / `ai/predictor.py` 변경 없음

- V3·V4 가중치는 `runs/segment/runs/segment/train_v{3,4}/weights/`에 보관, NAS 배포는 보류
- §27.4의 원인 분석을 근거로, 현재 시점에서는 추가 학습(V5)을 진행하지 않음

28.1 V5 재시도 시 고려사항 (보류, 참고용)

§27.4에서 확인된 "lr0=0.01 재시작 + 100epoch 장기학습으로 인한 mAP50-95 과적합"을 직접 해소하려면 아래 조합을 고려할 수 있음 (단, V1 수준 회복을 보장하지는 않음).

항목	현재	권장
lr0	0.01	0.001~0.002
cos_lr	False	True
epochs	100	40~50 (관찰된 정점 구간)
freeze	None	백본 일부 동결 검토